Hotel Revenue Manager

Requirements Specification and Data Blueprint

- **Project:** Hotel Revenue Manager, a web application for nightly rate decisions in small hotels
- **Repository:** https://github.com/stanleyreubens/565_project
- **Iteration:** 1 of the final project
- **Author:** Stanley Ogbumuo

---

1. Customer Statement of Requirements

1.1 The customer and the problem

The customer is the revenue manager of an independent hotel, or the person who ends up doing that job in a property too small to employ a specialist. In a chain hotel this work is supported by revenue management software that costs thousands of pounds a year. In a thirty room independent, it is done on a Sunday evening in a spreadsheet.

The decision itself is made many times a week. For a given night, and for each type of room the hotel sells, somebody has to choose a price. Set it too high and rooms go unsold, and an unsold room earns nothing at all, since the inventory expires at midnight. Set it too low and the hotel gives away margin on rooms that would have sold anyway.

Three pieces of information would improve that decision, and all three are available. Competitors publish their rates continuously on the online travel agencies. Events that will fill a city are announced months ahead. The hotel already knows its own costs and the group rates it has contracted. What is missing is anything that brings the three together at the moment the price is chosen. Today the manager checks a few competitor websites by hand, remembers that there is a conference in town, and makes a judgement.

1.2 Vision statement

Hotel Revenue Manager will be a web application that recommends a nightly rate for every room type on every date, and shows the manager the evidence behind each recommendation.

The system will gather competitor rates from the travel agencies, local events from a public events service, and the hotel's own cost floor and contracted group rates. It will combine these into a recommended price, and present that price alongside several named alternatives so that the manager can choose a position rather than accept a number. The recommendation will never fall below the hotel's cost floor or below a group rate already committed for those dates.

The product is a decision support tool and not an automated pricing engine. It will not connect to a booking system and it will not change a live rate. The manager reviews the recommendation, decides, and loads the chosen rate themselves. The application records what was recommended and what was actually charged, so that the quality of the recommendations can be judged over time.

1.3 Scope of the first iteration

The following will be delivered in iteration 1:

- Account registration and sign in, with hotels visible only to the user or organisation that owns them
- Property setup, covering hotels, their room types, and base rates
- A competitor set per hotel, with live rates retrieved for a chosen stay window

- A local events calendar with an editable impact score for each event
- The pricing recommendation itself, including the cost floor, the event uplift, and the alternative strategies
- Contracted group rates held by date range and applied as a floor
- A market report that charts the hotel's rate position against its competitors
- Reminders, so that rate reviews and contract renewals are not missed

1.4 Out of scope

Deliberately excluded from this iteration, and recorded here so that the boundary is explicit:

- Any write path into a property management system or channel manager. Rates leave the application as a recommendation on screen and nothing more.
- Payment processing, subscription billing, and anything else that would make this a commercial service.
- Forecasting from the hotel's own historical demand. The first iteration reacts to current competitor and event data rather than predicting from past occupancy.
- Machine learning. The pricing logic is a deterministic and inspectable calculation, which matters because the manager has to trust it.
- A mobile application. The web interface will work on a phone browser, but there will be no native app.
- Multi-currency conversion. Rates are handled in the currency the provider returns and are not converted.

---

2. Requirements Specification

Requirements are numbered for reference from the backlog. Priority uses MoSCoW: **M** must have for iteration 1, **S** should have, **C** could have if time allows.

2.1 Functional requirements

Accounts and access control

ID	Requirement	Priority
FR-1	A visitor shall be able to register with a name, an email address, and a password.	M
FR-2	Passwords shall be hashed before storage and shall never be returned by any endpoint.	M
FR-3	A registered user shall be able to sign in and receive a signed token for subsequent requests.	M
FR-4	A request carrying no token, an expired token, or an invalid one shall be refused, and the interface shall return the user to the sign in page.	M
FR-5	Where the email domain of a new account matches an existing organisation, the account shall join that organisation. Otherwise it shall stand alone.	S
FR-6	The system shall support three roles: standard user, organisation administrator, and platform administrator.	M
FR-7	A user who belongs to an organisation shall see every hotel that organisation owns. A user who does not shall see only the hotels they created.	M
FR-8	Signing out shall discard the stored token on the client.	M

Property setup

ID	Requirement	Priority
FR-9	A user shall be able to create a hotel record holding its name, address, city, star rating, and amenities.	M
FR-10	A user shall be able to edit a hotel, and to mark it inactive rather than deleting it.	M
FR-11	A user shall be able to define the room types a hotel sells, each with a base price, a capacity, and a list of features.	M
FR-12	A user shall be able to search a travel agency provider by property name and city, and import a matching property together with its room types where the provider returns them.	S

Competitor tracking

ID	Requirement	Priority
FR-13	A user shall be able to attach competitor properties to a hotel, recording the provider identifier, star rating, and distance.	M
FR-14	A user shall be able to remove a competitor from the set.	M
FR-15	The system shall retrieve live competitor rates for a stay window defined by check in date, check out date, number of adults, and number of rooms.	M
FR-16	Where the active provider supports it, the system shall show the price each travel agency is quoting for the same competitor property.	S
FR-17	The competitor data provider shall be selectable by configuration, without changing application code.	S
FR-18	When a provider is unreachable or returns an error, the interface shall say so plainly and shall continue to display the most recently stored figures.	M

Event demand

ID	Requirement	Priority
FR-19	The system shall retrieve events for a hotel's city across a chosen date range from a public events service.	M
FR-20	Each event shall carry a name, a venue, a start and end date, a category, its source, and an impact score from 1 to 10.	M
FR-21	A user shall be able to add an event by hand, and to override the impact score of any event.	M
FR-22	The interface shall present events on a calendar filtered by city and date range.	S

Pricing recommendations

ID	Requirement	Priority
FR-23	The system shall calculate a recommended nightly rate for a given hotel, room type, and date.	M
FR-24	The calculation shall take the average of the competitor rates for that date, and shall fall back to the room type's base price when no competitor rate is available.	M
FR-25	The calculation shall raise the recommendation by a demand multiplier derived from the total impact score of the events falling on that date.	M
FR-26	The recommendation shall never fall below a cost floor, defined as the cost per night plus a minimum margin.	M
FR-27	Where a contracted group rate covers the date and exceeds the cost floor, that rate shall become the floor instead.	M
FR-28	The system shall return alternative strategies alongside the recommendation.	M

ID	Requirement	Priority
	Balanced, Value Leader, and Occupancy Focused shall always be offered. Premium shall be offered for properties of four stars or more, and High Demand when the event multiplier is large enough to warrant it.	
FR-29	The interface shall show the inputs behind each recommendation, namely the competitor average, the events counted, and the floor that was applied.	M
FR-30	A user shall be able to record the rate actually loaded into their booking system, and the occupancy achieved, against that date.	S
FR-31	The system shall hold at most one pricing record per hotel, room type, and date.	M

Contracted group rates

ID	Requirement	Priority
FR-32	A user shall be able to record a group rate covering a date range for one room type, with a group name, a segment, and notes.	M
FR-33	A user shall be able to edit and delete a group rate.	M
FR-34	The system shall reject a group rate whose end date precedes its start date.	M
FR-35	The interface shall show contracted rates on a calendar for a chosen hotel.	S

Reporting

ID	Requirement	Priority
FR-36	The system shall produce a market report placing the hotel's rate against its competitor set across a date range.	S
FR-37	The report shall be charted, showing the hotel's position over time rather than a single figure.	S
FR-38	A user shall be able to review the history of recommended rate against actual rate and occupancy.	C

Reminders

ID	Requirement	Priority
FR-39	A user shall be able to create a reminder with a title, a due date, a priority, and an optional link to a hotel.	S
FR-40	A user shall be able to mark a reminder complete or dismiss it.	S
FR-41	The interface shall list reminders that are due soon separately from those already overdue.	S
FR-42	Reminders shall be visible to everyone in the owning organisation.	S

Administration

ID	Requirement	Priority
FR-43	Elevation to platform administrator shall be possible only through a maintenance script run against the database, never through the interface.	M
FR-44	A settings page shall report which competitor data provider is currently active.	C
FR-45	The service shall expose a health endpoint for deployment checks.	S

2.2 Non-functional requirements

Performance

ID	Requirement
NFR-	Requests served entirely from the database shall complete within 500 ms at the 95th percentile.

ID	Requirement
1	
NFR-2	Requests that call a third party shall complete within 5 seconds, and shall in no case exceed the 30 second execution limit imposed by the hosting platform.
NFR-3	A page shall become interactive within 3 seconds of navigation on a broadband connection.
NFR-4	A single refresh of competitor prices for one hotel shall issue no more than one provider request per competitor, so that free tier quotas are not exhausted.

Security

ID	Requirement
NFR-5	Passwords shall be hashed with bcrypt at a cost factor of 10 or greater.
NFR-6	Session tokens shall be signed with a secret held in the environment, and shall expire no later than 30 days after issue.
NFR-7	All traffic in the deployed environment shall be carried over HTTPS.
NFR-8	Credentials and API keys shall be supplied through environment variables. No secret shall be committed to the repository, and a documented example file shall list the variable names without their values.
NFR-9	Every query that reads hotel data shall be constrained by organisation or owner. A user shall not be able to read another tenant's data by supplying an identifier directly.
NFR-10	Database access shall go through an object relational mapper that parameterises queries, so that user input cannot alter their structure.
NFR-11	Cross origin requests shall be accepted only from the application's own front end.

Reliability and availability

ID	Requirement
NFR-12	The service shall be available 99 percent of the time across the assessment period.
NFR-13	Failure of a third party provider shall degrade the affected feature only. The rest of the application shall remain usable.
NFR-14	The database shall be backed up daily, and a restore shall be tested at least once during the iteration.
NFR-15	Errors shall be logged with enough context to identify the request, without recording passwords or tokens.

Usability and accessibility

ID	Requirement
NFR-16	A signed in user shall reach any principal screen in no more than two clicks from the dashboard.
NFR-17	The interface shall remain usable down to a viewport width of 375 pixels.
NFR-18	Text shall meet the WCAG 2.1 AA contrast ratio, form fields shall be labelled, and the application shall be operable by keyboard alone.
NFR-19	An error message shall state what went wrong and what the user can do about it.

Maintainability and portability

ID	Requirement
NFR-20	The database schema shall have a single definition from which the client and the migrations are both generated.
NFR-21	The same request handlers shall serve both local development and the deployed environment, so that behaviour does not diverge between them.
NFR-22	The application shall run on Node.js 20, and shall start with one command for the API and one for the front end.
NFR-23	Swapping the competitor data provider shall require a configuration change only.
NFR-24	The application shall support the current and previous major versions of Chrome, Firefox, Safari, and Edge.

Data handling

ID	Requirement
NFR-25	The API shall remain stateless, holding no session in memory, so that instances can be added or removed freely.
NFR-26	Dates shall be stored in UTC and rendered in the user's local time.
NFR-27	Monetary values shall be stored as numbers and never as formatted strings.
NFR-28	Only the personal data required to operate an account shall be stored, namely a name and an email address.

3. Data and Storage Blueprint

3.1 Data input

Data will reach the application by four routes. Three will be available to users and one will be restricted to maintenance.

Route 1: manual entry through web forms. This will be the primary route, and it will cover everything the hotel knows about itself. Hotels, room types and their base prices, competitor selections, contracted group rates, manually added events, impact score overrides, achieved occupancy, and reminders are all typed in by the user. Forms will validate before submission and the API will validate again on arrival, since a client side check protects the user's time but not the database.

Route 2: automated retrieval from third party web APIs. Competitor rates and local events are fetched over HTTPS as JSON, in response to a user action rather than on a schedule. Two categories of service are involved:

Source	Supplies	Called when	Key data returned
Travel agency rate provider (Booking.com through RapidAPI, Google Hotels through SerpApi, or HotelLens)	Live room rates for competitor properties	A user opens competitor analysis or requests a recommendation for a stay window	Property identifier, room description, nightly rate, currency, agency name
Ticketmaster Discovery API	Events in the hotel's city	A user opens the events calendar or prices a date range	Event name, venue, start and end dates, category, expected attendance

Responses will be validated and normalised on arrival, because provider payloads vary in shape and cannot be trusted to be complete. A missing or non numeric rate will be discarded rather than stored as zero, which would drag a competitor average downwards and produce a damaging recommendation. Provider credentials will be read from environment variables. Where a provider is unavailable, the failure will be caught and reported to the user, and previously stored figures will remain on screen.

Route 3: assisted import from a provider listing. A user will be able to search for their own property by name and city and import it, which will create the hotel record and, where the provider returns them, its room types. This route is in the design because typing twenty room types by hand is the point at which a user abandons the product. Imported records will be editable afterwards and will carry no special status.

Route 4: administrative scripts. Two operations will be kept out of the interface deliberately and performed by scripts run against the database: elevating an account to platform administrator, and backfilling organisation membership for accounts created before organisations existed.

Reading a dataset from an uploaded file, such as a CSV export from a property management system, is **not** part of this iteration. It is recorded in the backlog as a candidate for iteration 2.

3.2 Database and storage

Selected solution: PostgreSQL, a relational SQL database, hosted as a managed instance and accessed through the Prisma ORM.

The data is relational in the ordinary sense. A hotel has room types, a room type has priced nights, a priced night has the competitor observations that informed it, and every one of those belongs to an owner and an organisation. The queries the application will perform are joins across those relationships and comparisons over date ranges, which is what a relational engine is built to do. Three further properties decided the choice:

1. **Constraints belong in the database.** One priced night per hotel, room type, and date is a rule the schema can enforce by a unique constraint. Enforcing it in application code instead would eventually allow a duplicate through a second code path.
2. **Transactions.** Writing a pricing record together with the competitor prices that justify it must either happen completely or not at all. A half written recommendation is worse than none.
3. **Referential integrity.** Deleting a hotel must not leave orphaned room types and rates behind.

A document store such as DynamoDB or MongoDB was considered and rejected. The access patterns are not known in advance, since a market report may slice by date, by competitor, or by room type depending on what the manager is looking at, and a document store would require either denormalisation or a secondary index for each of those. The data volume for a small hotel is modest and does not call for horizontal partitioning. The application would gain nothing from a schemaless store and would lose the constraints described above.

Connection. The connection string will be supplied in the `DATABASE_URL` environment variable and will never be committed. One client instance will be shared across requests, cached in development so that reloading does not open a new connection pool each time. The schema file will act as the single source of truth, and both the typed client and the migrations will be generated from it.

3.3 Logical data model

The schema will define eleven tables and four enumerated types. Those that carry the domain:

Entity	Purpose	Notable fields
Organization	The tenant that owns hotels and users	Unique email domain
User	An account and its role	Email (unique), hashed password, role, organisation
Hotel	A property	Name, city, star rating, amenities, owner,

Entity	Purpose	Notable fields
		organisation, active flag
RoomType	A category of room sold by a hotel	Base price, capacity, features
Competitor	A rival property tracked against a hotel	Provider identifier, star rating, distance
Event	Something happening in a city that affects demand	Start and end dates, expected attendance, impact score, source
PricingHistory	One room type on one date	Recommended price, actual price, occupancy rate
CompetitorPrice	An observed competitor rate	Competitor name, room type, price, the pricing record it supports
SmerfRate	A contracted group rate	Date range, rate, group name, segment
Reminder	A dated task for the team	Due date, priority, status, optional hotel

Relationships:

- An organisation has many users, hotels, and reminders
- A user owns many hotels and raises many reminders
- A hotel has many room types, competitors, pricing records, group rates, and reminders
- A room type has many pricing records and group rates
- A pricing record has many competitor prices and relates to many events
- An event relates to many pricing records, since one conference affects every room type priced for that night

Constraints and indexes that the schema will enforce:

Rule	Mechanism
One pricing record per hotel, room type, and date	Unique constraint across the three columns
One account per email address	Unique constraint
One organisation per email domain	Unique constraint
Fast lookup of events for a city and date range	Composite index on city with start and end dates
Fast lookup of group rates covering a date	Composite index on hotel, room type, and the date range
Fast lookup of a team's reminders	Composite indexes on organisation with due date, and organisation with status

3.4 What is stored and what is not

Provider responses will not be archived in full. A rate retrieved for a competitor will be normalised and written as a `CompetitorPrice` row attached to the pricing record it informed, which preserves the evidence behind a recommendation without accumulating raw payloads. Everything the user types will be persisted. Session tokens will be held in browser storage on the client rather than server side, which is what allows the API to remain stateless. API keys will live in environment variables in the hosting platform, and in an untracked local file during development.

Appendix: Product Backlog

Item 4 of this assignment, the Agile product backlog, is maintained as tracked issues in the repository. Each user story below is logged as a separate issue under the **Issues** tab, labelled with its epic and carrying its acceptance criteria.

Repository issues: https://github.com/stanleyreubens/565_project/issues

The backlog holds **32 user stories** totalling **134 story points**, of which 83 points are must have work for iteration 1. Story points follow a Fibonacci scale and describe relative effort rather than hours.

ID	Epic	User story	Pts	Priority	Refs
US-01	Accounts and access	As a hotel owner, I want to register an account with my name, email, and a password	3	Must	FR-1, FR-2
US-02	Accounts and access	As a registered user, I want to sign in with my email and password	2	Must	FR-3
US-03	Accounts and access	As a signed in user, I want to be returned to the sign in page when my session is no longer valid	3	Must	FR-4, FR-8
US-04	Accounts and access	As a member of a hotel group, I want to have my account joined to my group automatically when I register with my work email	5	Should	FR-5
US-05	Accounts and access	As a organisation administrator, I want to see every property my organisation owns from one place	5	Must	FR-6, FR-7
US-06	Property setup	As a revenue manager, I want to create a hotel record with its address, star rating, and amenities	3	Must	FR-9
US-07	Property setup	As a revenue manager, I want to edit a hotel and mark it inactive rather than deleting it	3	Must	FR-10
US-08	Property setup	As a revenue manager, I want to define the room types my hotel sells with a base price and capacity	3	Must	FR-11
US-09	Property setup	As a revenue manager, I want to search for my property on a travel agency and import it	8	Should	FR-12
US-10	Competitor tracking	As a revenue manager, I want to attach competitor properties to my hotel	5	Must	FR-13, FR-14
US-11	Competitor tracking	As a revenue manager, I want to see what my competitors are charging for a stay window I choose	8	Must	FR-15
US-12	Competitor tracking	As a revenue manager, I want to see which travel agency is quoting which price for the same competitor	5	Should	FR-16
US-13	Competitor tracking	As a revenue manager, I want to be told clearly when competitor data cannot be retrieved	3	Must	FR-18
US-14	Competitor	As a platform administrator, I want to	3	Should	FR-17, FR-44

ID	Epic	User story	Pts	Priority	Refs
	tracking	switch the competitor data provider through configuration			
US-15	Event demand	As a revenue manager, I want to see the events happening in my city over the dates I am pricing	5	Must	FR-19, FR-20, FR-22
US-16	Event demand	As a revenue manager, I want to add an event the public listings do not carry	3	Must	FR-21
US-17	Event demand	As a revenue manager, I want to change the impact score of an event	2	Must	FR-21
US-18	Pricing recommendations	As a revenue manager, I want to see a recommended nightly rate for each room type on each date	8	Must	FR-23, FR-24
US-19	Pricing recommendations	As a revenue manager, I want to be offered several pricing strategies rather than one number	5	Must	FR-28
US-20	Pricing recommendations	As a revenue manager, I want to see the evidence behind a recommendation	5	Must	FR-29
US-21	Pricing recommendations	As a hotel owner, I want to never be shown a recommendation below my cost floor	5	Must	FR-26
US-22	Pricing recommendations	As a revenue manager, I want to record the rate I actually loaded and the occupancy I achieved	5	Should	FR-30, FR-31
US-23	Contracted group rates	As a revenue manager, I want to record a contracted group rate over a date range	5	Must	FR-32, FR-33, FR-34
US-24	Contracted group rates	As a revenue manager, I want to have a contracted rate act as the floor for the dates it covers	5	Must	FR-27
US-25	Contracted group rates	As a revenue manager, I want to see contracted rates laid out on a calendar	3	Should	FR-35
US-26	Reporting	As a revenue manager, I want to see my rate position against my competitor set over time	8	Should	FR-36, FR-37
US-27	Reporting	As a revenue manager, I want to review recommended against actual rates and the occupancy that followed	5	Could	FR-38
US-28	Reminders	As a revenue manager, I want to create a reminder with a due date and a priority	3	Should	FR-39
US-29	Reminders	As a revenue manager, I want to mark a reminder complete or dismiss it	2	Should	FR-40
US-30	Reminders	As a team member, I want to see what is due soon separately from what is already overdue	3	Should	FR-41, FR-42
US-31	Platform administration	As a platform administrator, I want to be elevated only by a script run against the database	2	Must	FR-43
US-32	Platform	As a platform administrator, I want to	1	Should	FR-45

ID	Epic	User story	Pts	Priority	Refs
	administration	check that the deployed service is alive			

Each story is written in the standard form, and the full text with acceptance criteria appears in the corresponding issue. The full text of every story is kept alongside this file in `Product-Backlog.md`.